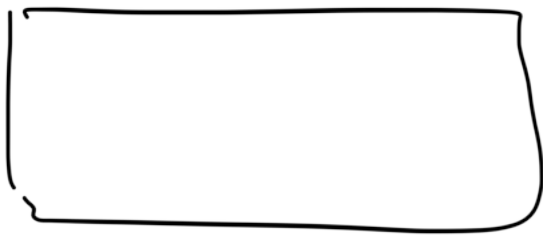


LLM
Week-5

L4 Tokenizer - Challenges

- we were talking transformers & masking.



↑ ↑ ↑ ↑
I am - -

word-wise
tokens.

- lang. modelling involves computing probab. over a seq. of tokens.

$$P(x_1) \prod_{i=2}^T P(x_i | x_1 \dots x_{i-1}) = P(x_1) P(x_2 | x_1) \dots$$

This is the reason why we have to split the input text into tokens in a consistent way. A simple method to use white space for splitting text.

Training & inference pipeline
tokenizer → token-ids → embeddings
↓
ids to tokens ← lang. models

- ## L2 Motivaⁿ for sub-word tokenizaⁿ

- ↕ 2 diff. entremes

— almost
∞ lokam
(500 K)

- embedding materialize, probab. calculⁿ, parameter size $\Rightarrow$ problems in 500k.
- if we use charac. based then we have to ^{find} very little info in our context window.
- $\therefore$, sub-words level tokenizaⁿ

do
is
ing
n't

make this 1 token

also, by doing this
the size of vocabs is
moderate.

this is
always
more
preferred in
LLMs

- 3 types $\rightarrow$ charac.
 - $\rightarrow$ sub-words
 - $\rightarrow$ words
- $\rightarrow$ byte pair
 - $\rightarrow$ word piece
 - $\rightarrow$ sentence piece

L3 BPE (Byte Pair Encoding)

- our wishlist
 - $\rightarrow$ mod. size vocabs
 - $\rightarrow$ efficiently handle out-of-vocab words
 - $\rightarrow$ should be lang. agnostic

- while building input $\rightarrow$ normalization
(lower space) $\rightarrow$ pre-tokenization (based on whitespace) $\rightarrow$ tokenization (sub-words level)

- algo. $\rightarrow$

1. start with dict. with words & count
2. add symbol $\langle w \rangle$ at end of each word
3. set req. no. of merges (HPT)
4. start a charac. - freq. table (base vocab)
5. get the freq. count for a pair of charac.
6. merge pairs with max occurrence.

knowing	—	3	} unique words = 13
the	—	1	
man	—	1	

$\Downarrow$

k	—	3	$\Rightarrow$ initial vocab size = 22
n	—	13	
o	—	9	
$\langle w \rangle$	—	16	

$\swarrow$

k n — 3
 n o — 3
 o w — 3

adjust
 the
 vocab
 freq.

k
 n
 :
 in

find the
 highest
 freq., add
 that pair to
 vocab

we have to now think how many
 merges should be allowed? $\Rightarrow 32k$!

$\hookrightarrow$ exactly that would be your
 (+100) vocab size
 initial

- the final vocab has initial vocab &
 all the merges (in order). The vocab
 words are broken down into 2 or
 more subwords.

- fertility rate

knowing — knowing
 $\hookrightarrow$ fertility rate is 2.

- at test time, the input word is
 split into a seq. of characters & the
 characters are merged into a larger
 known symbols.

- BPE helps in non-segmented lang.
 In practice, we can use lang.-specific

morphology based word segmenters.
Then that will be the input for
BPE algorithm.

Eq. $V = \{a \dots z, \text{lo}, \text{he}\}$

input $\rightarrow$ hello lol

h e l l o l o l

$\downarrow$ lo

h e l lo lo l

$\downarrow$ he

he #l #lo lo #l

14 Word-Piece Tokenizer

- in bpc, we merged a pair of tokens with highest frequency.
- we assign a score for every candidate

$$\text{score} = \frac{\text{count}(\alpha, \beta)}{\text{count}(\alpha) \text{count}(\beta)}$$

this score will be used specially to break the ties.

- Now, we select those pairs, individual tokens are less, but when they merge

it occurs many times. This was the pt of improvement over BPE.

charac. $\xrightarrow{N}$ 30k - 50k
vocab merges medium
cut. vocab.

you can do reverse as well,

(larger vocab) $\xrightarrow{N}$ medium
breaks cut. vocab.

LS Sentence Piece Tokenizer

- a word can be broken into segments by several ways, as per the vocab. given to me.

$v = h, e, l, l, o, h, e, e, l, l, o, u$

hello $\Rightarrow h, e, l, l, o$

$v = h, e, l, l, o, e, l, h, e, l, l, o, u$

hello $\Rightarrow h, e, l, l, o$

- same vocab, just change its order gives diff. segments.
- BPE $\rightarrow$ greedy & deterministic.

- The probabilistic approach is to find the subword seq. $x^* \in \{x_1, \dots, x_n\}$ that maximizes the likelihood of x .

$$P(x) = \prod_{i=1}^n P(x_i)$$

$$\sum_{x \in V} P(x) = 1$$

for all seq. in dataset, we define the likelihood funcⁿ as -

$$L = \sum_{s=1}^{|D|} \log(P(x^s))$$

$$= \sum_{s=1}^{|D|} \log\left(\sum_{x \in S(x^s)} P(x)\right)$$

Expectaⁿ - Maximizaⁿ algo.

- Sentence piece

1. create a large vocab V using BPE or extended suffix array!
2. $E \Rightarrow$ estimate the probab. for every token in given vocab
3. $M \Rightarrow$ use viterbi to segment the corpus & return optimal segments that maximises the log likelihood.

4. compute the likelihood for each new subword from optimal segment
 5. remove bottom $k\%$ which has least likelihood.
 6. iterate the process, until you get the desired no. of tokens.
- Eg. whereby $\Rightarrow$ how viterbi algo. can be used to prune & then you don't have to use the entire probabilities.